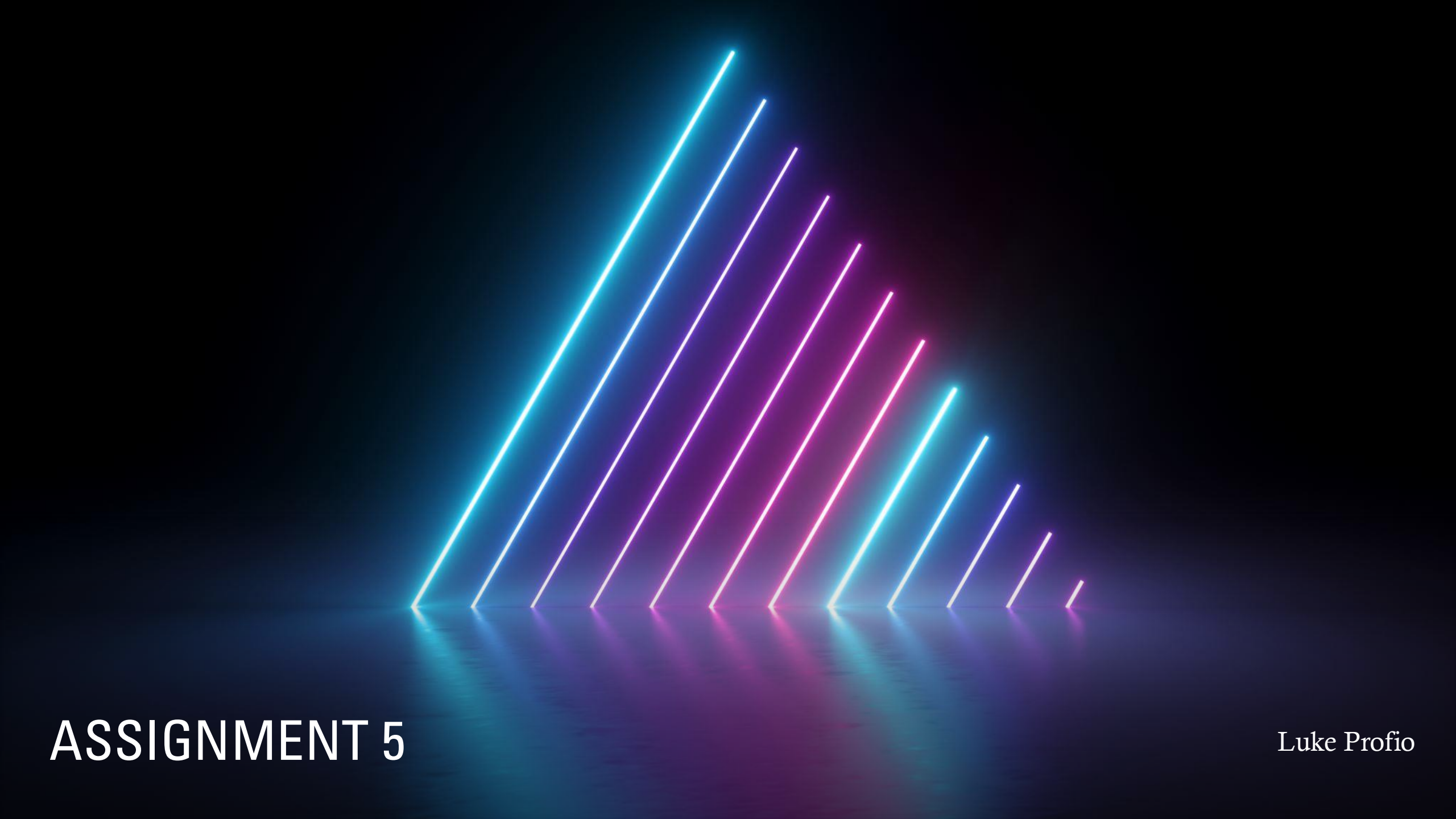


ASSIGNMENT 5

Luke Profio

DOWNSTREAM TASK 1: IN-HOSPITAL MORTALITY PREDICTION [LOGISTIC REGRESSION / RANDOM FOREST]

- Our objective is to use only static admission features to predict mortality for each hospital stay.
- Features include the age of the patient at admission, admission type (e.g. emergency vs. other), and gender.
- A static method helps to establish a simple baseline, and shows how minimal demographics/admission context can already carry signal.

DOWNSTREAM TASK 1: DATA PREPARATION

- To prepare the data, we convert categorical columns (e.g. admission type, gender) into separate indicator variables.
- The feature set is then generated, including this indicators along with age.
- The data is then split into training and testing sets (80/20).
- A one-hot encoding ensures models can handle non-numeric data.
- Splitting into a training and test set helps prevent overfitting and provides an accurate performance estimate.

DOWNSTREAM TASK 1: DATA PREPARATION

```
df = pd.get_dummies(df, columns=['ADMISSION_TYPE','GENDER'])
static_features = df[['age','ADMISSION_TYPE_EMERGENCY','GENDER_F','GENDER_M']]
```

✓ 0.0s

```
X_train, X_test, y_train, y_test = train_test_split(
    static_features, labels, test_size=0.2, random_state=42)
```

✓ 0.0s

DOWNSTREAM TASK 1: MODEL TRAINING

- Next up, we train two models: logistic regression and random forest.
- Both models are evaluated on the testing set using ROC-AUC and overall accuracy.
- It is ideal to compare a linear and non-linear ensemble model to see which better captures patterns in the static features.
- Complementary metrics include AUC for ranking ability, and accuracy for overall correctness.
- Both models have the same AUC, but logistic regression has a slightly improved accuracy, making it the model of choice.

DOWNSTREAM TASK 1: MODEL TRAINING

```
    models = {
        'LogisticRegression': LogisticRegression(max_iter=1000),
        'RandomForest': RandomForestClassifier(n_estimators=100)
    }
    for name, model in models.items():
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        auc = roc_auc_score(y_test, model.predict_proba(X_test)[:,1])
        print(f"{name}: AUC={auc:.3f}, Accuracy={accuracy_score(y_test,preds):.3f}")
9]  ✓  0.9s

LogisticRegression: AUC=0.695, Accuracy=0.904
RandomForest: AUC=0.686, Accuracy=0.904
```

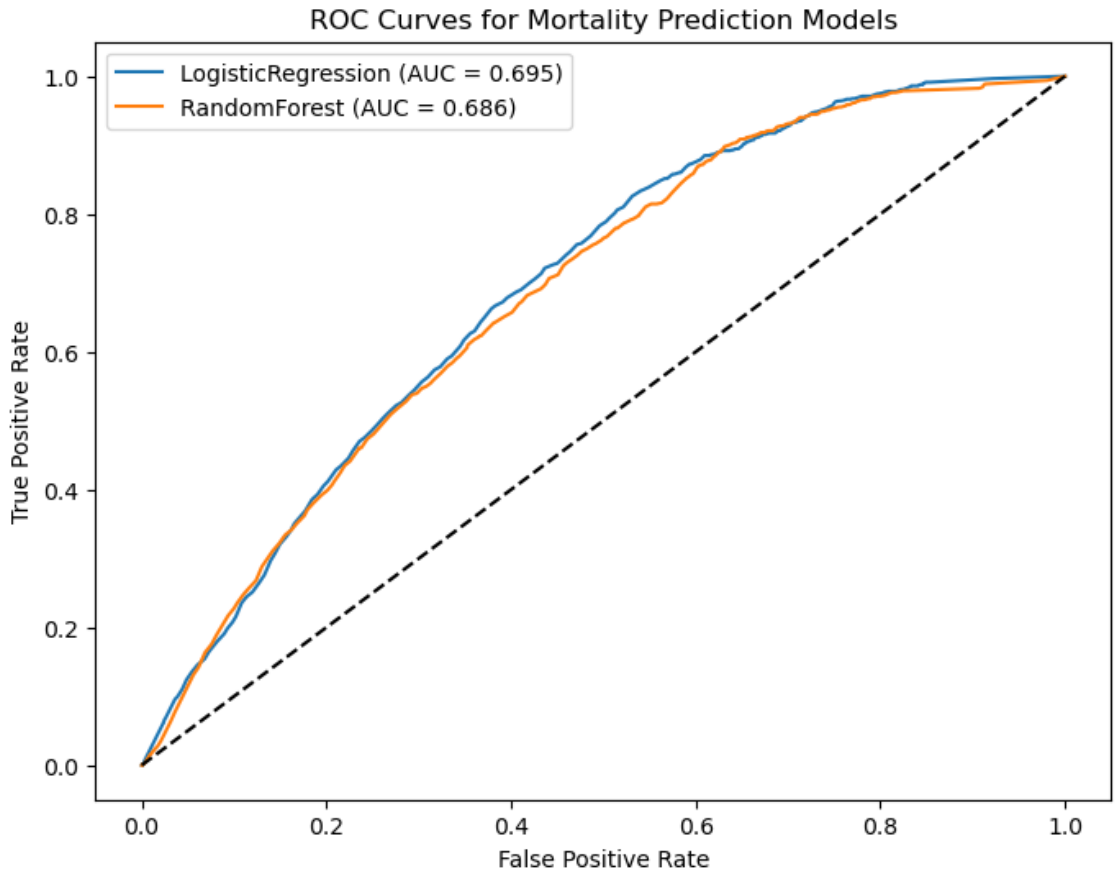
DOWNSTREAM TASK 1: MODEL EVALUATION

- We present the ROC-AUC and accuracy scores for both models.
- This highlights which model performs better on each metric.
- These benchmarks help guide decisions on whether more complex features or methods are needed.
- As outlined earlier, since logistic regression and random forest have comparable AUC scores, logistic regression is the model of choice given its higher accuracy.

DOWNSTREAM TASK 1: MODEL EVALUATION

	precision	recall	f1-score	support
0	0.90	1.00	0.95	10658
1	0.00	0.00	0.00	1138
accuracy			0.90	11796
macro avg	0.45	0.50	0.47	11796
weighted avg	0.82	0.90	0.86	11796

DOWNSTREAM TASK 1: MODEL EVALUATION



DOWNSTREAM TASK 1: EVALUATION

- Additionally, we perform a grid search on random forest's key configurations (e.g. number of trees, maximum depth).
- We report on the best combination of parameters and detail the corresponding classification metrics (e.g. precision, recall).
- Optimizing models improves generalization without over- or under-fitting.
- Age and the emergency admission type are the most important features.

DOWNSTREAM TASK 1: EVALUATION

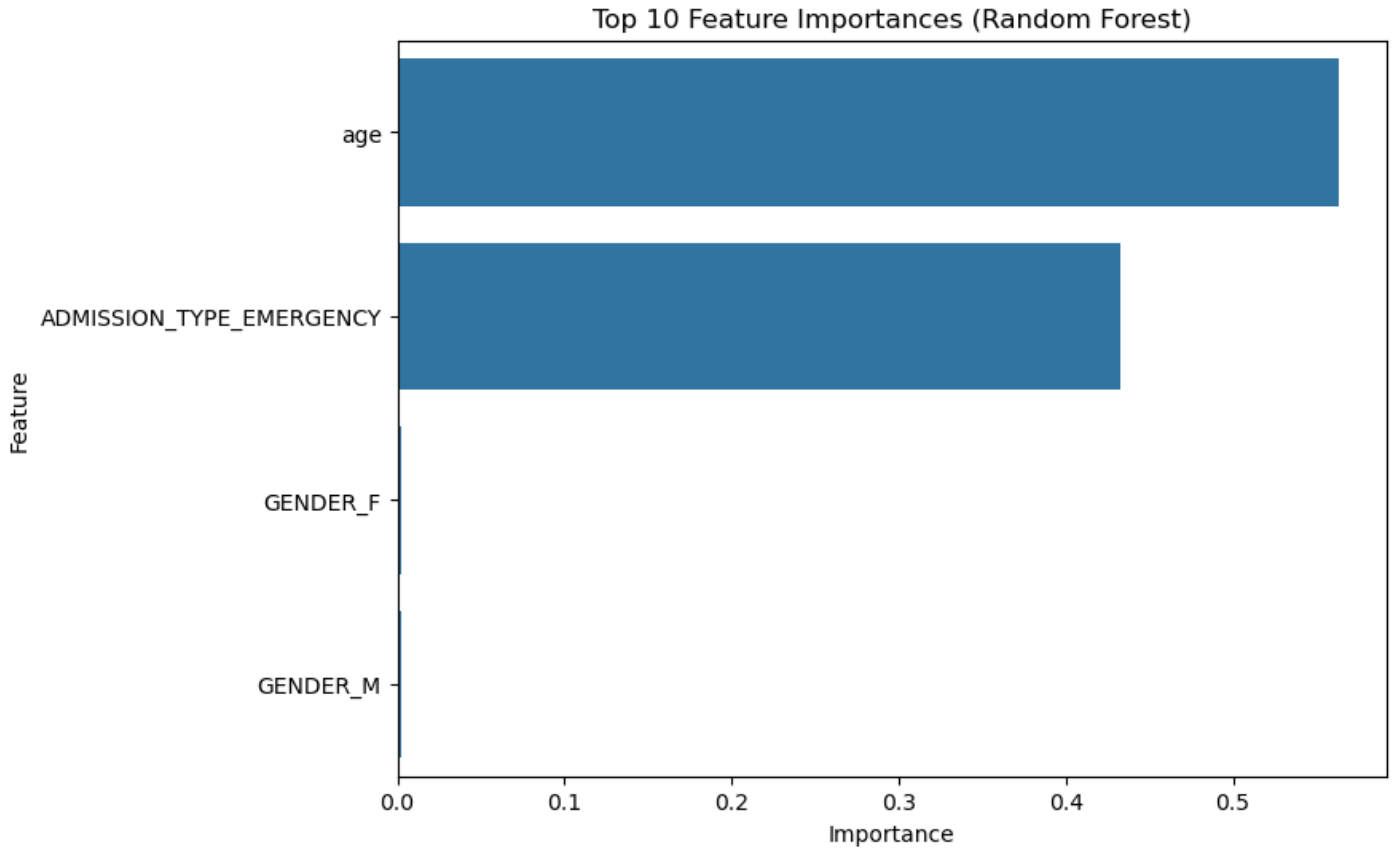
```
# Feature importance for Random Forest
import seaborn as sns
import pandas as pd

importances = best.feature_importances_
feat_names = X_train.columns
feat_imp = pd.Series(importances, index=feat_names).sort_values(ascending=False).head(10)

plt.figure(figsize=(8,6))
sns.barplot(x=feat_imp.values, y=feat_imp.index)
plt.title("Top 10 Feature Importances (Random Forest)")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()

param_grid = {'n_estimators':[100,200], 'max_depth':[4,8]}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=3)
grid.fit(X_train, y_train)
best = grid.best_estimator_
y_pred = best.predict(X_test)
print(classification_report(y_test, y_pred))
```

DOWNSTREAM TASK 1: EVALUATION



DOWNSTREAM TASK 1: EVALUATION

	precision	recall	f1-score	support
0	0.90	1.00	0.95	10658
1	0.00	0.00	0.00	1138
accuracy			0.90	11796
macro avg	0.45	0.50	0.47	11796
weighted avg	0.82	0.90	0.86	11796

DOWNSTREAM TASK 2: DIAGNOSIS CATEGORY PREDICTION [LSTM]

- In this second downstream task, given the sequence of diagnosis-category IDs for a single hospital admission (in the order they occurred), we predict what the next diagnosis category will be.
- Additionally, we demonstrate the application of sequence modeling (LSTM) to temporal EHR data.
- This goes beyond static features, capturing the order and progression of diagnoses.

DOWNSTREAM TASK 2: DATA PREPARATION

- First, we map raw ICD-9 codes into 18 broad numeric categories.
- All codes are grouped for a single admission into a time-ordered sequence.
- Each admission's sequence is split into "history" (all but last) as input and "last" as the target label.
- Each input sequence is padded or truncated to a fixed length of 50.
- Finally, we one-hot encode the 18 target categories.
- The practice of binning reduces thousands of codes into a more manageable set.
- Sequence slicing creates a standard next-step prediction format, while padding helps to ensure a uniform input size for the LSTM.

DOWNSTREAM TASK 2: DATA PREPARATION

```
# These bins carve continuous ICD-9 codes into 18 broad ranges.
bins   = [0,140,240,280,290,320,390,460,520,580,630,680,710,740,760,780,800,1000,2000]
labels = list(range(len(bins) - 1))  # → [0,1,…,17], one label per bin
```

✓ 0.0s Python

```
os.chdir('/Users/led7283981/Downloads/mimic-iii-clinical-database-1.4')
base_path = os.getcwd()
diag = pd.read_csv(os.path.join(base_path, 'DIAGNOSES_ICD.csv.gz'))

# Extract first three digits, replace non-numeric with 999, then map into our 18 bins
diag['code3'] = (
    diag['ICD9_CODE']
        .str.extract(r'(^\d{3})')[0]
        .fillna('999')
        .astype(int)
)
diag['cat'] = pd.cut(diag['code3'], bins=bins, labels=labels).astype(int)
```

✓ 0.5s Python

```
# Sort so that SEQ_NUM gives the order within each admission
diag = diag.sort_values(['SUBJECT_ID', 'HADM_ID', 'SEQ_NUM'])

# Group by admission and collect the category sequence
seqs = diag.groupby('HADM_ID')['cat'].apply(list)
```

✓ 0.6s Python

DOWNSTREAM TASK 2: DATA PREPARATION

```python
# For each admission with ≥2 diagnoses:
# - X = all categories except the last
# - y = the final category
X_seqs = [s[:-1] for s in seqs if len(s) > 1]
y_cats = [s[-1]  for s in seqs if len(s) > 1]
```

✓ 0.0s Python

```python
maxlen = 50
X_pad = pad_sequences(
    X_seqs,
    maxlen=maxlen,
    padding='pre',      # pad at the front
    truncating='pre'    # if too long, keep the last 50
)
```

✓ 0.0s Python

```python
num_classes = len(labels)        # 18 categories
y_cat       = to_categorical(y_cats, num_classes=num_classes)
```

✓ 0.0s Python

DOWNSTREAM TASK 2: MODEL ARCHITECTURE

- We use an Embedding layer to learn 32-dimensional dense vectors for each category ID.
- This is followed with a single LSTM layer of 64 units to process the sequence.
- Finally, we utilize a Dense + softmax layer over 18 categories.
- This embedding approach captures relationships between diagnosis categories.
- LSTM is tailored for sequential dependencies, where order matters.
- The softmax output gives a probability distribution for the next category.

DOWNSTREAM TASK 2: TRAINING AND VALIDATION

- We split the data into a training and test set (80/20).
- Within training, we reserve 10% for validation at each epoch.
- We train for 10 epochs using categorical cross-entropy loss and the Adam optimizer.
- Training vs. validation loss and accuracy are tracked each epoch.
- Validation curves help in indicating over- or under-fitting early on.
- Multiple epochs enable to model to refine its internal sequence representations.

DOWNSTREAM TASK 2: TRAINING AND VALIDATION

```python
X_train, X_test, y_train, y_test = train_test_split(
    X_pad, y_cat,
    test_size=0.2,
    random_state=0
)
```

✓ 0.0s Python

```python
model = Sequential([
    Embedding(input_dim=num_classes, output_dim=32, input_length=maxlen),
    LSTM(64),
    Dense(num_classes, activation='softmax')
])
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
model.summary()
```

✓ 0.0s Python

DOWNSTREAM TASK 2: TRAINING AND VALIDATION

```python
history = model.fit(
    X_train, y_train,
    validation_split=0.1,  # use 10% of train as a val set
    epochs=10,
    batch_size=128
)

loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {acc:.4f}")
```

7] ✓ 1m 8.0s Python

```
Epoch 1/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 19ms/step - accuracy: 0.4254 - loss: 2.0980 - val_accuracy: 0.4397 - val_loss: 1.8668
Epoch 2/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 6s 19ms/step - accuracy: 0.4411 - loss: 1.8644 - val_accuracy: 0.4433 - val_loss: 1.8453
Epoch 3/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 21ms/step - accuracy: 0.4434 - loss: 1.8423 - val_accuracy: 0.4502 - val_loss: 1.8303
Epoch 4/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 21ms/step - accuracy: 0.4441 - loss: 1.8391 - val_accuracy: 0.4414 - val_loss: 1.8289
Epoch 5/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step - accuracy: 0.4518 - loss: 1.8159 - val_accuracy: 0.4455 - val_loss: 1.8195
Epoch 6/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step - accuracy: 0.4525 - loss: 1.8035 - val_accuracy: 0.4465 - val_loss: 1.8166
Epoch 7/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step - accuracy: 0.4469 - loss: 1.8186 - val_accuracy: 0.4502 - val_loss: 1.8105
Epoch 8/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step - accuracy: 0.4460 - loss: 1.8097 - val_accuracy: 0.4499 - val_loss: 1.8053
Epoch 9/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 21ms/step - accuracy: 0.4463 - loss: 1.8048 - val_accuracy: 0.4489 - val_loss: 1.8061
Epoch 10/10
330/330 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step - accuracy: 0.4514 - loss: 1.7995 - val_accuracy: 0.4502 - val_loss: 1.8048
Test accuracy: 0.4521
```

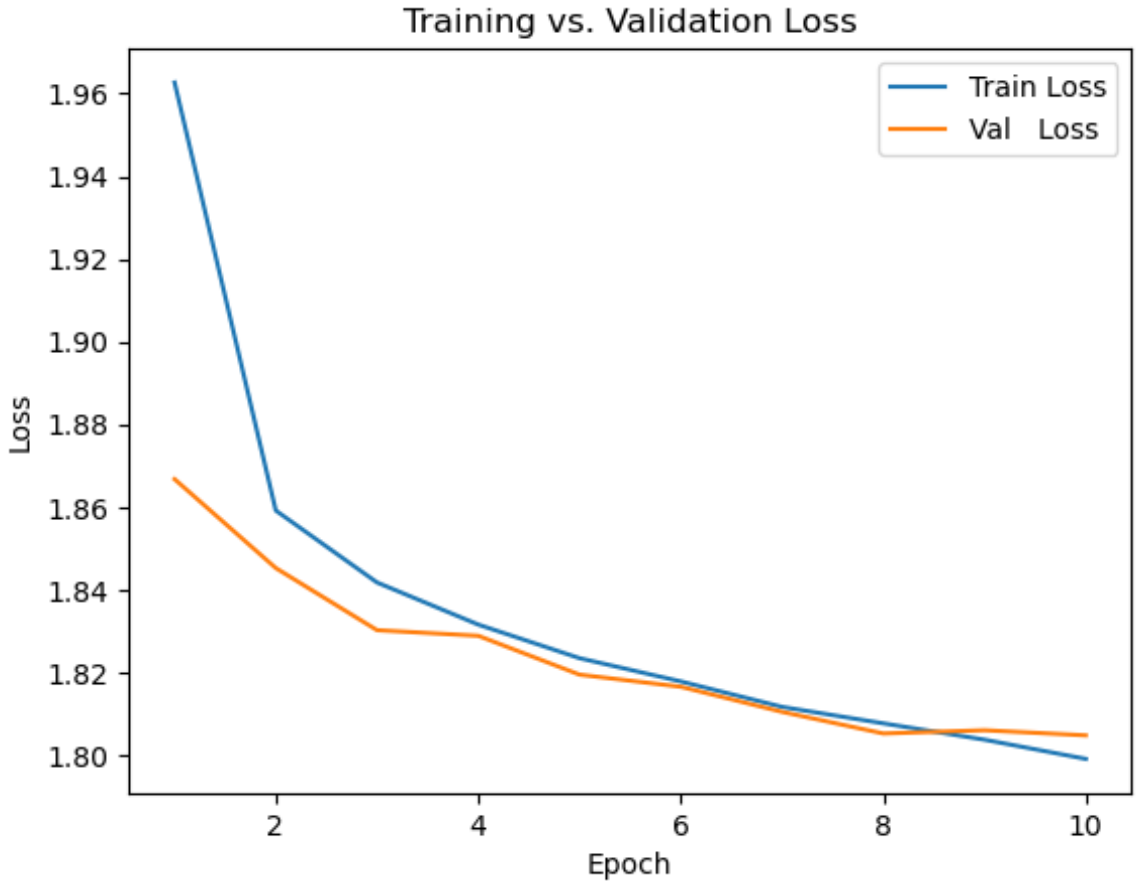
DOWNSTREAM TASK 2: EVALUATION

- As a part of evaluation, we measure final test accuracy on held-out admissions.
- The training/validation loss and accuracy curves are plotted to assess convergence.
- The distribution of true next categories in the test set is also plotted.
- Finally, we generate a confusion matrix to see which categories are most frequently mis-predicted.
- Test accuracy is the primary marker of overall performance.
- Each of the curves plotted check model health (e.g. stable learning).
- The distribution and confusion plots provide insights on where the model struggles.

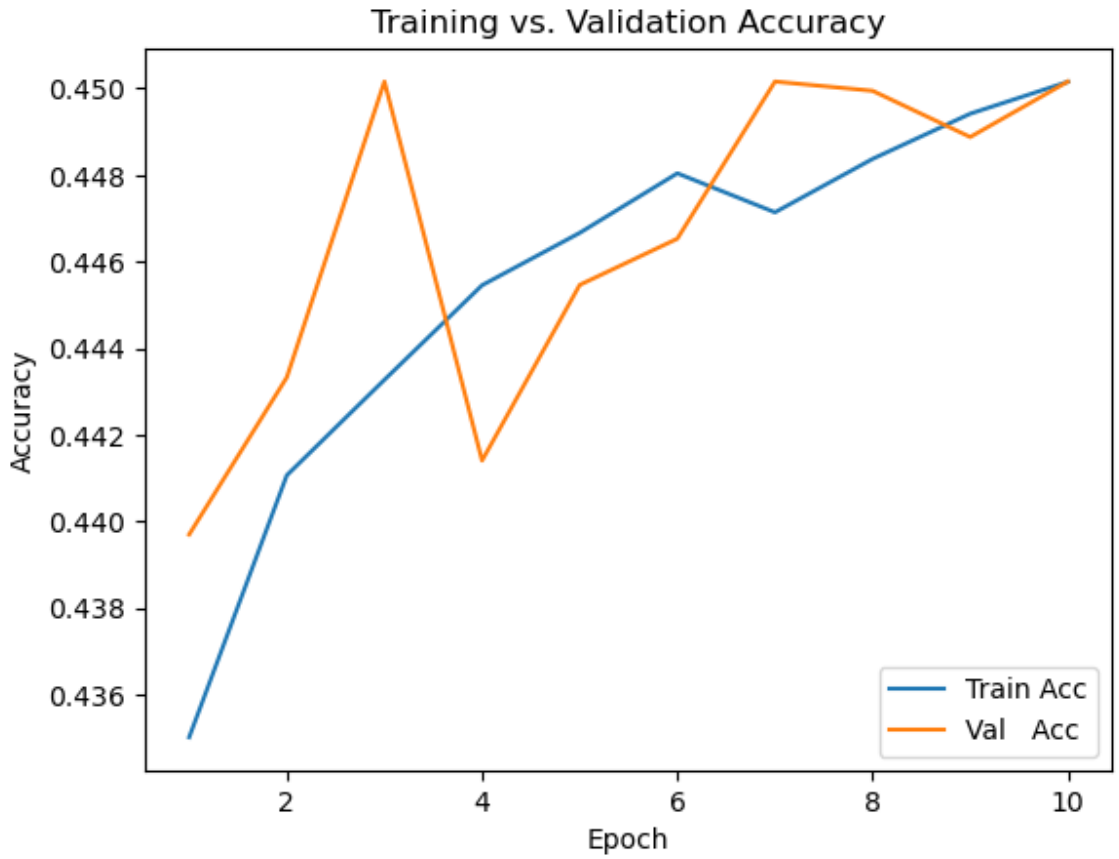
DOWNSTREAM TASK 2: EVALUATION

- Over time, the training and validation loss significantly decrease. Most of this decrease is attained by epoch 4.
- Over epoch 1-10, accuracy increases by a little over 1%.
- These evaluations indicate that core changes are required on the LSTM model (e.g. architecture, hyperparameters, data preparation), in order to increase accuracy further.

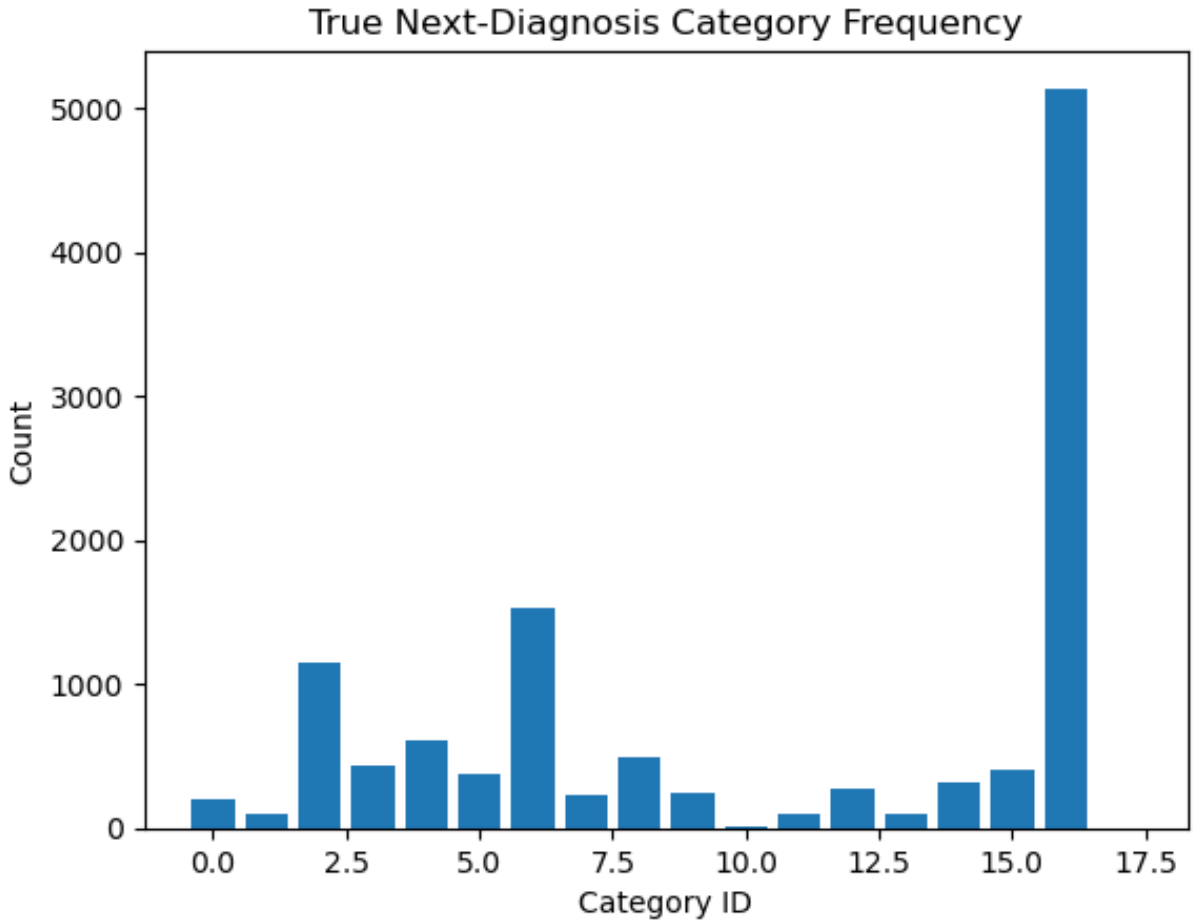
DOWNSTREAM TASK 2: EVALUATION



DOWNSTREAM TASK 2: EVALUATION



DOWNSTREAM TASK 2: EVALUATION



DOWNSTREAM TASK 2: EVALUATION

